As we have seen in the previous sub-section, string similarity is used by spell-checkers to decide the best suggestion to correct typos.

But recently another even more important use case has become popular: bioinformatics, matching sequences of DNA. This is a computational-intensive task, so using the wrong data structures can make it impossible to solve.

When we have to compare just two strings, then computing directly the Levenshtein distance is the most effective way to go; if, however, we have to compare a single string to `n` other strings to find the best match, computing `n` times the Levenshtein distance becomes impractical. The running time, in fact, would be `O(n * m * M)`, where `m` is the length of our search string and `O(M)` is the average length of the `n` strings in the corpus.

Turns out, we can do much better using a trie: by using the same algorithm shown for spell-checkers (without the threshold-based pruning), computing all the distances will only take time `O(m * N)`, where `N` is the total number of nodes in the trie, and while the trie construction could take up to `O(n * M)`, it would only happen once at the beginning, and if the rate of lookups is high enough, its cost would be amortized.

In theory `N` here can be `O(n * M)`, as we have seen in section 6.2, if no two strings in the corpus share the same prefix. In practice, however, it is likely that `N` is order of magnitudes smaller than `n * M`, and closer to `O(M)`. Moreover, as we have seen in the last sub-section, if we set a threshold for the max tolerance, i.e. for the largest difference